

6. 임베딩

토큰화: 입력된 문장을 컴퓨터가 이해할 수 있는 단위인 토큰으로 나누는 과정



텍스트 벡터화(Text Vectorization)

- 텍스트를 컴퓨터가 이해할 수 있는 숫자로 변환하는 과정

원-핫 인코딩

- 문서에 등장하는 각 단어를 고유한 색인 값으로 매핑한 후, 해당 색인 위치를 1로, 나머지 위치는 모두 0으로 표시하는 방식

빈도 벡터화

- 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식
- ex) apples라는 단어가 총 4번 등장 → 해당 단어의 벡터값은 4
- 벡터 형태로의 변환이 간단하지만, 벡터의 **희소성**이 크다 → **차원의 저주** 문제 발생!
- 텍스트의 벡터가 입력 텍스트의 의미를 내포하지 않아 두 문장이 의미적으로 유사하더라도 벡터의 유사도와는 무관할 수 있다. → 해결 방안으로 등장한 것이 **워드 임베딩**

언어 모델

- 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델
- 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측을 수행

P(만나서 반갑습니다.) = 0.081		P(서울특별시) = 0.59	
Input : 안녕하세요	...	Input : 대한민국 ____ 강남구	...
P(아니오, 괜찮습니다.) = 0.000001		P(아이스 아메리카노) = 0.000001	

그림 6.1 언어 모델

- 주어진 문장 뒤에 나올 수 있는 문장이 매우 다양해 확률 계산이 어려움

⇒ 이러한 문제를 해결하기 위해 문장 전체 대신 하나의 토큰 단위로 예측하는 방법: **자기회귀 언어 모델**

자기회귀 언어 모델

- 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측
- 언어 모델에서 조건부 확률: 이전 단어들의 시퀀스가 주어졌을 때, 다음 단어의 확률을 계산하는 것

수식 6.1 언어 모델의 조건부 확률

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

- 이전에 등장한 모든 토큰 정보를 고려하며 문장의 문맥 정보를 파악하여 다음 단어를 생성

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = P(w_1)P(w_2 | w_1) \dots P(w_t | w_1, w_2, \dots, w_{t-1})$$

- 6.1 수식에 조건부 확률의 연쇄법칙을 적용
- 조건부 확률의 연쇄법칙: 하나의 사건이 일어날 확률을 다른 사건들의 조건부 확률을 이용해 계산하는 방식
- 문장 전체의 확률은 첫 번째 단어의 확률 $P(w_1)$ 과 각 단어가 이전 단어들의 조건부 확률에 따라 발생할 확률의 곱으로 나타낸다.

통계적 언어 모델

- 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성 및 분석
- 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률을 예측
- 마르코프 체인을 이용해 구현

- 마르코프 체인(Markov Chain) : 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측하는 빈도 기반의 조건부 확률 모델



빈도 기반 조건부 확률의 문제점

- 빈도 기반의 조건부 확률 모델에서는 주어진 데이터에서 각 변수가 발생한 빈도수를 기반으로 확률을 계산
 - 단어의 순서와 빈도에만 기초하므로 문맥을 제대로 파악하지 못하면 부적절한 결과 생성
 - 처음 등장한 단어나 문장에 대해 정확한 예측X

⇒ **데이터 희소성 문제** 발생 : 관측한 적 없는 데이터를 예측하지 못하는 문제

하지만 통계적 언어 모델은 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포를 생성

- 대규모 자연어 데이터 처리에 효과적

N-gram

- 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정
- 입력 텍스트를 하나의 토큰 단위가 아닌 N개의 토큰을 묶어 분석
 - N개의 단어 → 하나의 단위로 취급
 - N = 1 → 유니그램(Unigram)
 - N = 2 → 바이그램(Bigram)
 - N = 3 → 트라이그램(Trigram)
 - N ≥ 4 → N-gram

Unigram : 안녕하세요 만나서 진심으로 반가워요	안녕하세요. 만나서. 진심으로. 반가워요
Bigram : 안녕하세요 만나서 진심으로 반가워요	안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요
Trigram : 안녕하세요 만나서 진심으로 반가워요	안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

그림 6.3 N이 1, 2, 3인 N-gram

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

N-gram 언어 모델에서 t번째 토큰의 조건부 확률을 계산하는 수식

- w_t : 예측하려는 단어
- w_{t-1} 부터 w_{t-N+1} : 예측에 사용되는 이전 단어들
- 이전 단어들의 개수를 결정하는 N 값을 조정하여 모델의 성능을 조절한다.

N-gram 구현

```

import nltk

def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)

sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = ngrams(sentence, 1)
bigram = ngrams(sentence, 2)
trigram = ngrams(sentence, 3)

print(unigram)
print(bigram)
print(trigram)

unigram = nltk.ngrams(sentence.split(), 1)
bigram = nltk.ngrams(sentence.split(), 2)
trigram = nltk.ngrams(sentence.split(), 3)

print(list(unigram))
print(list(bigram))
print(list(trigram))

```

```

... [('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]

```

- 파이썬 코드로 구현한 N-gram과 NLTK 라이브러리에서 지원하는 N-gram 함수는 동일한 출력값을 반환
- N-gram은 작은 규모의 데이터셋에서 연속된 문자열 패턴 분석과 관용적 표현 분석에 효과적
- 단어의 순서가 중요한 자연어 처리 작업 및 문자열 패턴 분석에 활용

TF-IDF

- 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법
- 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적인 가중치
- 즉, BoW(Bag-of-Words)에 가중치를 부여하는 방법!

Bow

- 문서나 문장을 단어의 집합으로 표현하는 방법
- 문서나 문장에 등장하는 단어의 중복을 허용해 빈도를 기록함

예를 들어, ['That movie is famous movie', 'I like that actor', 'I don't like that actor'] 말뭉치를 BoW로 벡터화하면 표 6.2와 같다.

표 6.2 BoW 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

- BoW로 벡터화하면 모든 단어는 동일한 가중치를 가짐

단어 빈도(Term Frequency, TF)

- 문서 내에서 특정 단어의 빈도수를 나타내는 값

수식 6.5 단어 빈도

$$TF(t, d) = count(t, d)$$

- TF 값이 높으면 →
 - 해당 단어가 특정 문서에서 중요한 역할
 - 단어 자체가 전문 용어나 관용어일 수도 있음

문서 빈도(DF)

- 한 단어가 얼마나 많은 문서에 나타나는지를 의미

수식 6.6 문서 빈도

$$DF(t,D) = count(t \in d: d \in D)$$

- DF 값이 높으면 → 특정 단어가 많은 문서에서 등장, 일반적으로 널리 사용되며 중요도가 낮은 단어
- DF 값이 낮으면 → 특정 단어가 적은 수의 문서에만 등장, 특정한 문맥에서만 사용되며 중요도가 높은 단어

역문서 빈도(IDF)

- 전체 문서 수를 문서 빈도로 나눈 다음에 로그를 취한 값
- 문서 내에서 특정 단어가 얼마나 중요한지를 나타냄

수식 6.7 역문서 빈도

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

TF-IDF

- 문서 빈도와 역 문서 빈도를 곱한 값
- 문서 내에 단어가 자주 등장 & 전체 문서 내에 해당 단어가 적게 등장 → TF-IDF는 커짐
- 전체 문서에서 자주 등장할 확률이 높은 관사나 관용어 → 가중치 낮아짐

TF-IDF 클래스

```
tfidf_vectorizer = sklearn.feature_extraction.text.TfidfVectorizer(
    input = "content",
    encoding = "utf-8",
    lowercase=True,
    stop_words=None,
    ngram_range=(1,1),
    max_df=1.0,
    min_df=1,
    vocabulary=None,
    smooth_idf=True,
)
```

- **input** : 입력될 데이터의 형태
 - 기본값 content: 문자열 데이터 혹은 바이트 형태의 입력값
 - file : 파일 객체
 - filename : 파일 경로
- **encoding** : 바이트 혹은 파일을 입력값으로 받을 경우 사용할 텍스트 인코딩 값
- **lowercase** : 소문자 변환 여부
- **stop_words** : 불용어, 분석에 도움이 되지 않기 때문에 단어 사전에 추가되지 않음
- **ngram_range** : 사용할 N-gram의 범위 (최솟값, 최댓값)
 - (1,1): 유니그램
 - (1, 2): 바이그램
- **max_df** : 최댓값 문서 빈도, 전체 문서 중 일정 횟수 이상 등장한 단어는 불용어로 처리
- **min_df** : 최솟값 문서 빈도, 전체 문서 중 일정 횟수 미만으로 등장한 단어를 불용어 처리
- **vocabulary** : 미리 구축한 단어사전이 있을 경우 해당 단어 사전 사용
- **smooth_idf** : IDF 분모 처리

TF-IDF 계산


```

from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "That movie is famous movie",
    "I like that actor",
    "I don't like that actor"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix = tfidf_vectorizer.transform(corpus)

print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)

```

```

[[0.         0.         0.39687454 0.39687454 0.         0.79374908
  0.2344005 ]
 [0.61980538 0.         0.         0.         0.61980538 0.
  0.48133417]
 [0.4804584  0.63174505 0.         0.         0.4804584  0.
  0.37311881]]
{'that': 6, 'movie': 5, 'is': 3, 'famous': 2, 'like': 4, 'actor': 0, 'don': 1}

```

- **toarray** : TF-IDF를 넘파일 배열로 변환, (문서 수)x(단어 수)
- **vocabulary_** 속성: TF-IDF에 사용된 단어사전, 키는 고유한 단어 값은 해당 단어에 대한 색인 값 의미

⇒ 문서마다 중요한 단어만 추출할 수 있으며 벡터값을 활용해 문서 내 핵심 단어를 추출할 수 있다.

하지만 빈도 기반 벡터화는 문장의 순서나 문맥을 고려하지 않아서 순서가 중요한 작업에는 부적합!